
COOLPilot
self-checking cameras, measured on AWS Graviton

COOLPilot: technical report

What it is. A camera-health workbench for ADAS camera fleets. OpenCV 5 checks every frame of every camera against that camera's own normal. An agent verifies suspicions with more OpenCV 5 tools before an alarm stands. A diagnostic autopilot tunes the detector, but nothing it proposes ships without fresh evidence and a human. It runs on AWS Graviton.

What it is not. It is not a safety system, and it is not validated on real lens dirt or rain. All faults in the evidence are synthetic, except for the independent DRIVE-C corruptions.

- **Live (read-only):** https://n7ebpehiuh2hczk2uzsherigia0pyqfg.lambda-url.us-east-1.on.aws/
- **Code:** `github.com/azharizz/coolpilot`

Every number below links to an evidence file in `docs/evidence/`.

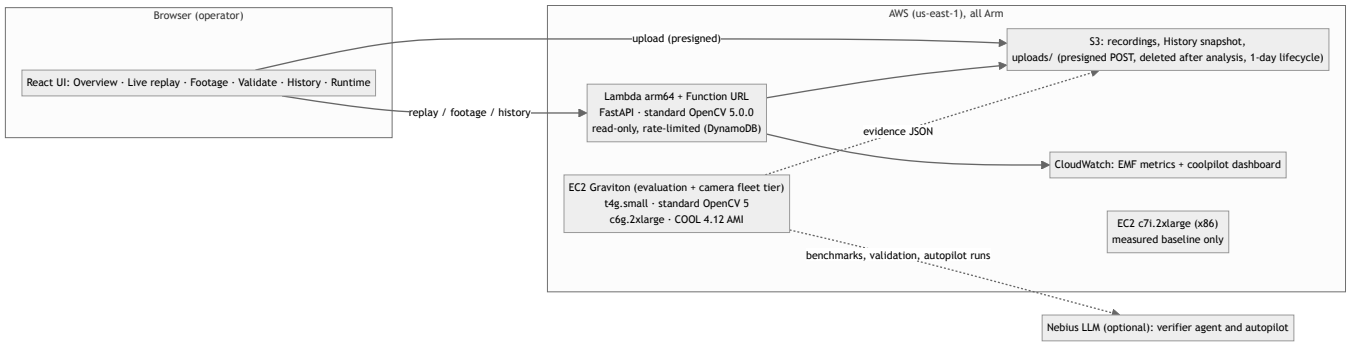
1. Problem

A degraded camera rarely fails loudly: a smear, glare, a dark tunnel or a frozen frame still delivers frames. The starting detector used absolute thresholds, and they misfired on real scenes. A bright sky is always "clipped", so glare fired constantly.

Starting point, run #63: - **15 of 53** gate checks failed. - **1,332 false alarms per hour.**

Evidence: `evidence/run63-before.json`.

2. Architecture



- **Public endpoint:** serverless (Lambda arm64), using the standard OpenCV 5.0.0 wheel.
- **COOL:** ships as a Marketplace AMI, which Lambda cannot run. It therefore runs on EC2 Graviton, where evaluation and the camera-fleet tier also run.
- **x86:** appears only as a measured baseline. Nothing in the deployed system runs on x86.

3. How it works

Detector (`coolpilot/camera_health.py`). Each frame is resized and converted with OpenCV, and detail is measured at 640 px. It is split into 4×4 tiles. Each tile gets: - brightness (`mean`) - clipping (`compare` + `countNonZero`) - detail (variance of `Laplacian`, via `meanStdDev`)

The frame thumbnail also feeds the freeze check (`absdiff`).

Rules compare each tile with the camera's own references, learned only while the view is healthy: glare, underexposure, blur, obstruction (the same tiles staying low while the rest keeps detail), blackout and freeze. They also apply hysteresis and persistence. A camera that is dark from its first frame is treated as its own normal (night).

Loop A, runtime verifier (`coolpilot/verify.py`, `verify_agent.py`). An obstruction suspicion triggers three OpenCV 5 tools: - `inspect_texture`: HSV sky / low-texture check - `detect_road_users`: NanoDet on the **OpenCV 5** `cv2.dnn` **new engine** - `measure_motion`: Farneback optical flow

A fixed rule (the default) or an LLM agent then **dismisses**, **confirms** or **escalates to a human**. It is fail-safe: any error keeps the alarm, and deciding before measuring is refused.

Loop B, diagnostic autopilot (`experiments/autopilot.py`). An LLM agent tunes bounded detector settings using OpenCV-measured tools: - `evaluate`, `explain_failure` (per-frame rule trace), `paired_compare` (fault vs the same clean frames), `ablate` - it ends with `propose`, which stays **pending human approval** and is validated once on never-inspected clips before anything changes

Diagram and step-by-step traces: AGENT_WORKFLOW.md.

4. Results

What	Before	After	Evidence
Fault checks failed, unseen cameras (detector)	12 / 18	5 / 18	`evidence/calibration/`
False alarms/h, 42 never-inspected clips	232 (detector)	**0 with the verifier**, no real fault lost (11/36 failed either way)	`evidence/aws/verify-*fresh*.json`
Same, paced in real time on Graviton	233.8	**0.0**, +17% CPU	`evidence/aws/paced-fresh-*.json`
Two more never-inspected Oxford sets (fresh2, fresh3)	n/a	4/18 failed, 0 false alarms/h with the verifier	`evidence/tuning/validation3/`, `validation4/`
DRIVE-C test (independent benchmark; **seen** since one failed validation)	day 13/60, night 0/90	day **28/60**, night **29/90**	`evidence/tuning/validation2/`, `validation3/`
OpenCV 5 new DNN engine vs 4.x classic (NanoDet)	96.0 ms (4.13)	**71.9 ms** (Graviton t4g); 14.7 vs 56.0 ms at 8 threads (c6g)	`evidence/aws/dnn-*.json`, `evidence/cool/dnn-*.json`

Joint tuning on Oxford + DRIVE-C dev (`BEFORE_AFTER.md` 6b–6c): - The first freeze **failed** validation: DRIVE-C test day dropped 13 → 0/60. The two choices behind it were dropped, and the reasons are recorded. - Self-calibrating limits and the autopilot's best proposal looked better on tuning data (77 → 101/150), then **added false alarms on unseen clips** (0 → 32.5/h). The approval gate kept them out. - What shipped: detail measured at 640 px, a 1.2 s obstruction hold, and the night policy.

5. COOL on AWS Graviton

The COOL setup, from `COOL.md`: - **AMI:** `COOL-graviton2-v2` (`ami-009ab0cb3e3e0acc1`) on a **c6g.2xlarge** (Neoverse-N1, 8 vCPU). - **COOL build:** OpenCV 4.12.0 with KleidiCV 0.5.0 + Carotene, ARMPL 25.07, TBB.

COOL ran the core workload: - the app's own runtime check reported `distribution: COOL` - all 62 tests passed - the service ran under it - measurements and alarms were identical to the standard 4.12 wheel

Capacity, measured (8 parallel camera streams, production pipeline; on-demand prices from the AWS Pricing API):

Instance and build	Frames/s	Real-time cameras	CPU	Analysis p95	$ per 1,000 camera-hours
c6g.2xlarge, COOL 4.12	387	27	98.9%	18.5 ms	10.07
c6g.2xlarge, OpenCV 5.0	440	30	98.8%	21.6 ms	**9.07**
c7i.2xlarge (x86), OpenCV 5.0	484	33	98.6%	20.0 ms	10.82

Decision: standard OpenCV 5 on Graviton. Arm is 16% cheaper per camera than x86. COOL brings no gain for this pipeline, for three reasons: - decoding is about a third of the CPU, and it is outside COOL's acceleration - our heaviest calls are not on the KleidiCV list - this COOL image is OpenCV 4.12, without OpenCV 5's DNN engine

Reproduce with `experiments/cool_benchmark.sh` and `experiments/cool_capacity.sh`.

6. Limits and failures (stated, not hidden)

- **Lens dirt and occlusion:** 0/6 on Oxford holdouts, 0/25 on DRIVE-C. The tile statistics cannot separate it from scene change without false alarms.
- **Night:** 29/90. One of the 3 clean night scenes still alarms. A camera dark from power-on is not flagged; the report says "verify".
- **Glare:** 1–2 of 3 per holdout. **Mild defocus and underexposure:** missed.
- **The autopilot did not beat random search** at the same budget. The LLM verifier is worse than the fixed rule (99.6 vs 0 false alarms/h) and not reproducible across runs, so the rule is the default.
- **Data:** one Oxford drive plus DRIVE-C's 10 scenes, all synthetic faults. Five daytime dev scenes were too few to select rules, which we learned from the failed validations.

7. Reproducibility and responsible operation

- **Pinned dependencies:** `requirements.txt`, with OpenCV 5.0.0.93 and NumPy 2.5.3.
- **Tests:** 62, passing on Mac, Graviton (standard and COOL) and x86.
- **Re-running the evaluations:** every script is in `experiments/` and `deploy/aws/`, with commands in the README and `BEFORE_AFTER.md`. Datasets are content-hashed. Held-out data is never read during tuning, and every time that rule was bent (a seen test split) it is labelled.
- **Uploads:** footage is deleted right after analysis. Users must confirm they have the right to upload.
- **The public site:** read-only and rate-limited. LLM keys are never sent to benchmark subprocesses.
- **Evidence:** per-run JSON, CloudWatch metrics and agent traces.

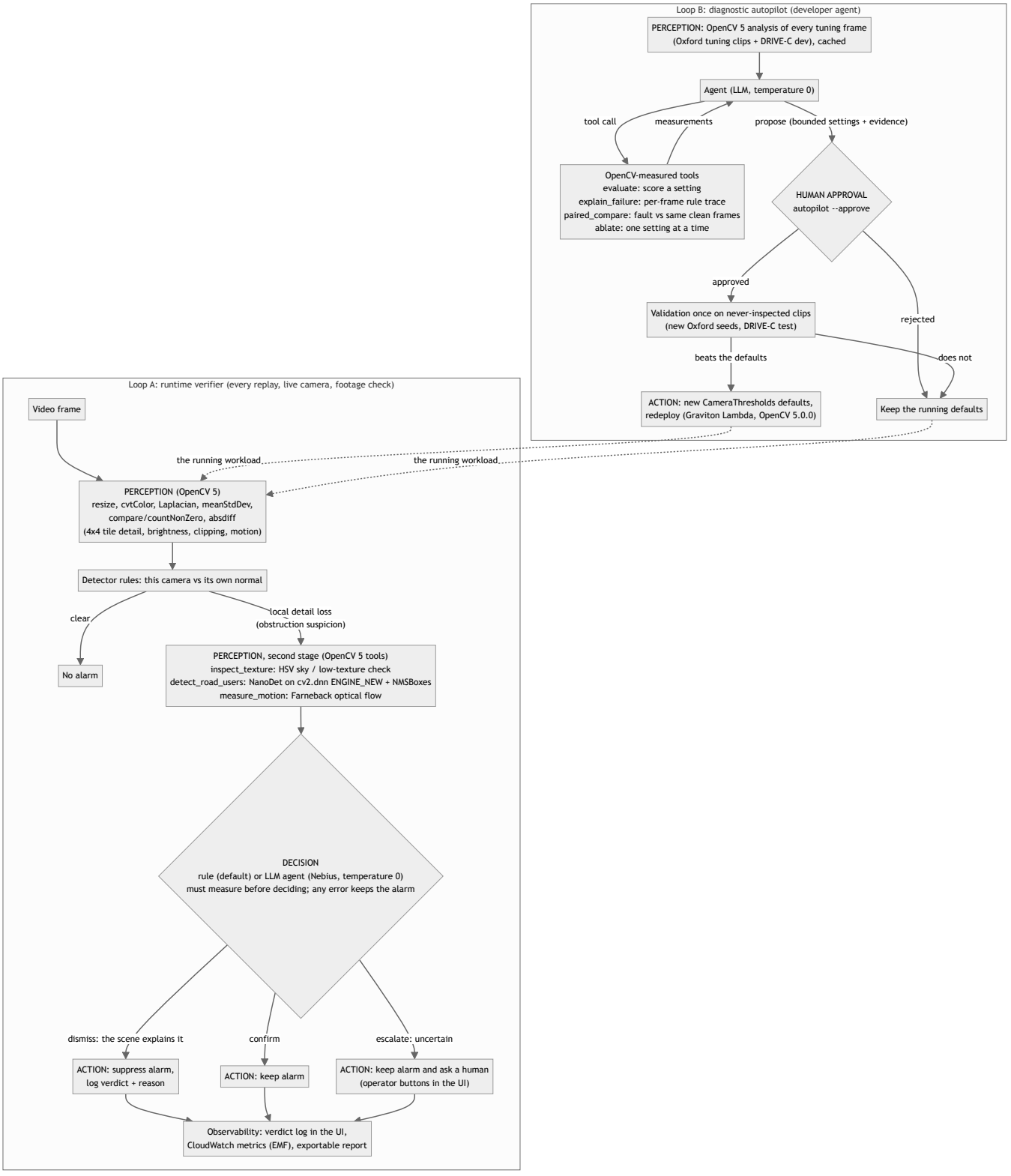
Agent workflow: perception, decision, action

COOLPilot has two agent loops. In both, OpenCV 5 measurements decide what happens next:

- **Loop A, runtime verifier:** every live replay, live camera session and footage check.
- **Loop B, diagnostic autopilot:** a developer agent that tunes the detector.

Every number below comes from a recorded run; the files are listed with each trace.

Diagram



Where it lives: - **Loop A:** `coolpilot/camera_health.py` (perception and rules), `coolpilot/verify.py` (tools and rule decision), `coolpilot/verify_agent.py` (LLM decision), `coolpilot/objects.py` (NanoDet on the OpenCV 5 DNN engine). - **Loop B:** `experiments/autopilot.py`. The earlier configuration agent, which also searches processing settings, is `coolpilot/agent.py`.

Trace A: the autopilot changes its next call because of OpenCV output

Recorded on 2026-10-25, Graviton t4g.small, OpenCV 5.0.0. Budget: 16 evaluations. Wall time: 189.5 s. File: `docs/evidence/autopilot/tools.json`, field `trace`.

Step	Tool call	OpenCV-measured result	What it changed next
0	`evaluate({})` (current defaults)	77 of 150 DRIVE-C dev fault clips caught, 0 false alarms, Oxford 0 failed. Weakest: defocus 3/25, low light 11/25	Starts diagnosing a weak class instead of guessing
1	`explain_failure("S01_defocus_blur_s1")`	No alarm. Per-frame log detail ratio around onset swings between -0.34 and +0.17, while this camera's learned normal swing is about 0.17	Asks what the faults actually do to the measurements
2–5	`paired_compare` for defocus, lens occlusion, low light, underexposure	Defocus s1–s3 keep **0.97–1.00** of clean detail (invisible), s5 keeps 0.41. Low light s1 is only **0.84×** as bright. Underexposure s1 **0.90×**. Occlusion: the 4th-lowest tile drops to 0.16 at s5	Mild darkening is the gap: brightness drops of 10–20% are missed by the fixed 0.65 ratio. So it tests the self-calibrating limit
6	`ablate("adaptive_k", [0, 1, 2, 3, 4])`	k=0: **77**, 0 FA · k=1: 86, **31 FA** · k=2: 86, **31 FA** · k=3: **101, 0 FA** · k=4: 91, 0 FA	Picks k = 3 and then checks the floor next to it
7	`ablate("adaptive_floor_dark", [0.05 … 0.25])`	0.05: 62, 62 FA · 0.10: 84, 31 FA · 0.15: 82, 31 FA · **0.20: 101, 0 FA** · 0.25: 95	Keeps floor 0.20 and moves on to the blur floor
8	`ablate("adaptive_floor_blur", …)`	Used the last 4 evaluations (16 of 16), then hit the limit on its 5th value. A bug in our tool discarded those 4 results and returned only "Evaluation budget exhausted". The tool now returns partial results	The agent never saw the blur-floor measurements
9–17	`ablate("adaptive_floor_blur", …)` ×9	"Evaluation budget exhausted. Call propose."	Wasted turns: the agent repeated the call instead of proposing
18	`propose({adaptive_k: 3, floors 0.20 / 0.40, …})`	Scored outside the budget: **101/150**, 0 false alarms, low light 23/25, underexposure 21/25, defocus 9/25	Written as **pending human approval**, so nothing is applied

What happened after the proposal (failure handling and human control): - The proposal was validated once on **fresh2**, Oxford clips nobody had inspected. Verifier-checked false alarms rose from **0 to 32.5 per hour**: the self-calibrated blur limit fired on glare's washed-out detail. On DRIVE-C test, day clips dropped from 28 to 18 of 60. - It was **never approved**, so the running site kept its defaults. - A narrower follow-up (self-calibrating brightness, blur never looser) **tied** the defaults on fresh3, a second never-inspected set, so it was not shipped

either. Details: `docs/BEFORE_AFTER.md` section 6c. - Honest comparison at the same budget: the `evaluate`-only agent reached 102/150, and random search reached 103, 100 and 83. **The diagnostic tools did not beat random search.** The value shown here is the loop itself (OpenCV output, then the next call, then the human gate, then validation), not a better optimiser.

Reproduce: `python -m experiments.autopilot --cache cache.pkl --arm tools --budget 16 --out tools.json` (needs `NEBIUS_*` in the environment), then `python -m experiments.autopilot --approve tools.json`.

Trace B: the verifier decides with OpenCV evidence, at one recorded moment

Recorded on Graviton, OpenCV 5.0.0, on the never-inspected `oxford-camera-holdout-fresh` clips (unpaced, every frame). File: `docs/evidence/aws/verify-oxford-camera-holdout-fresh-validation.json`, clip `left_s1101_glare` at **4.06 s**. Ground truth at that moment: **no fault**. The glare in this clip starts later.

Step	Loop A stage	Recorded value
1	Perception: tile statistics	Tile 1's detail fell below 35% of its own recent normal while the rest of the view kept its detail, for 0.5 s (the obstruction rule at that time). The **obstruction suspicion** fires on tile 1
2	Tool `inspect_texture` (HSV)	Tile 1 relative texture 0.812, brightness **1.72×** the median tile, saturation **0.1**: open sky
3	Tool `detect_road_users` (NanoDet, `cv2.dnn` new engine)	Road users cover **0%** of tile 1 (the limit is 40%)
4	Tool `measure_motion` (Farneback)	Tile 1 flow **0.0 px**, other tiles median **5.75 px**: the tile is static
5a	Decision, **rule** (default)	**dismiss**: "Suspected tiles [1] show scene that cannot reveal lens loss (open sky now, or never detailed); the loss is the scene, not the lens." Latency 0.21 s
5b	Decision, **LLM agent** (same evidence)	**escalate** to a human. It quoted every measurement above ("open sky… scene-driven, not lens-related… 0% coverage… 0.0 px flow while other tiles 5.75 px…") yet still asked for a person. Tool order: `inspect_texture` → `detect_road_users` → `measure_motion` → `decide`. Latency 6.2 s
6	Action	Rule: the alarm is suppressed and the verdict logged. Agent: the alarm is kept and the operator's Confirm / Dismiss buttons appear

The same comparison over all 43 suspicions on these 42 never-inspected clips:

Arm	False alarms/h	Fault checks failed	Decisions	Median latency
No verifier	232.4	11/36	n/a	n/a
Rule (default)	**0.0**	11/36	43 dismiss	0.13 s
LLM agent	99.6	11/36	25 dismiss, 18 escalate	2.9 s

- 41 of the 43 suspicions were scene (sky), not lens.
- The rule also dismissed 2 suspicions that fell inside a real obstruction interval. Obstruction detection is 0 of 6 with or without the verifier, so nothing was lost there, but the verifier cannot rescue it either.
- The agent is the human-in-the-loop option: slower, more cautious, and it escalates instead of silencing.
- Paced in real time on Graviton (`docs/evidence/aws/paced-fresh-{off,rule}.summary.json`): 233.8 → **0.0** false alarms/h, no real fault lost, +17% CPU.

Reproduce: `python -m experiments.verify_eval --dataset oxford-camera-holdout-fresh --split validation --arms none rule agent --out verify.json`.

Evaluation summary (Agentic Vision evidence)

Criterion	Evidence
Task success	Loop A removes the dominant false-alarm cause: 232 → 0 per hour on 42 never-inspected clips, and 233.8 → 0 paced on Graviton, with identical fault detection. Loop B reached 101/150 on tuning data but failed unbiased validation (see failure handling)
Failure handling	Loop A is fail-safe: any provider or tool error keeps the alarm; `decide` before any measurement is refused; reasons over 600 characters are truncated. Loop B: two tuning proposals failed validation and were rejected (`BEFORE_AFTER.md` 6b–6c); budget exhaustion and invalid tool calls return errors instead of acting
Observability	Per-decision verdict, reason, tools and latency (UI verdict log and JSON); CloudWatch EMF metrics and a `coolpilot` dashboard; full autopilot traces in `docs/evidence/autopilot/`
Human control	Loop A: escalations surface Confirm / Dismiss for an operator. Loop B: proposals stay `pending_human_approval` until `--approve`; shipping a setting with a cost (the night policy) was a user decision
Honest limits	The LLM verifier is worse than the fixed rule (99.6 vs 0 false alarms/h) and not reproducible across runs. The autopilot did not beat random search. Neither loop solves lens occlusion

Running COOLPilot on COOL (migration guide)

Status: **not run on COOL yet.** The code and deploy scripts are prepared, but this account's AWS Free Plan cannot subscribe to paid Marketplace products. Upgrading the account (or using another one) and subscribing are the operator's steps. Nothing in this repository claims a COOL measurement until the checklist at the end has been completed.

What COOL is (from the AWS Marketplace listings)

- OpenCV's Cloud Optimized OpenCV Library: OpenCV 5 built for AWS Graviton with KleidiCV plus additional parallelization tuning. The Graviton4 Docker edition also links Arm Performance Libraries (ARMPL), for ECS/EKS.
- Delivered as Ubuntu 24.04 AMIs, one product per Graviton generation (G2, G3, G4).
- 7-day free trial, then a usage-based software fee of about $0.01–$0.04 per hour on top of EC2 (for example, G2 on c6g.2xlarge $0.02/h; G4 on c8g.xlarge $0.01/h and m8g.4xlarge $0.04/h). Check the listing for current prices.
- Recommended instances: G2 c6g.2xlarge or larger; G4 m8g.4xlarge or larger. c8g, m8g and r8g are compatible.
- Vendor support: support@opencv.org.

Layout on the AMI:

What	Path
Python environments	`/opt/cool/venvs/python_3.10`, `python_3.11`, `python_3.12` (activate with `source /opt/cool/venvs/python_3.12/bin/activate`)
Python packages	`PYTHONPATH=/opt/cool/python_3.12/site-packages/`
Shared libraries	`LD_LIBRARY_PATH=/opt/cool/cpp_sdk/lib`
C++ CMake config	`OpenCV_DIR=/opt/cool/cpp_sdk/lib/cmake/opencv4`

How COOLPilot uses it

- `deploy/aws/remote_setup.sh cool` runs the app with the vendor interpreter (`$COOL_VENV/bin/python`, default `/opt/cool/venvs/python_3.12`). Our other dependencies are installed with `pip --target ~/deps-cool` from `requirements.txt` **minus opencv and numpy**, so the standard wheels can never shadow the vendor `cv2`. The systemd unit exports `PYTHONPATH`, `LD_LIBRARY_PATH=/opt/cool/cpp_sdk/lib` and `COOLPILOT_COOL_ROOT`.
- `coolpilot/runtime.py` reports `distribution: "COOL"` only when the loaded `cv2` module lives under `COOLPILOT_COOL_ROOT` (default `/opt/cool`) on Linux ARM64. It also records which

acceleration layers `cv2.getBuildInformation()` mentions (KleidiCV, Carotene, ARMPL, NEON). Every experiment stores this environment, so a COOL result can be told apart from a standard one.
- `threads` (1, 2, 4) is part of the agent's search space, so the agent can find where COOL's parallel tuning pays off on a given instance size instead of assuming it does.

What to expect (measured facts, before COOL)

Measured on EC2 t4g.small (Graviton2, 2 vCPU) with the standard `opencv-python-headless` 5.0.0 wheel (`results/profile-standard.json` from `remote_setup.sh`), CPU seconds per 271-frame clip:

Stage	CPU s
Video decode	~1.9
OpenCV analysis (resize, cvtColor, Laplacian, meanStdDev, compare, countNonZero)	~0.7
Python alarm logic	~0.2

Two consequences:

1. **The standard aarch64 wheel already reports KleidiCV.** KleidiCV in the build information is therefore not evidence of COOL; the module path and the AMI are.
2. **COOL can only speed up part of our workload.** Analysis is about a quarter of the per-frame CPU. Decoding dominates, and it runs in FFmpeg, which COOL is not documented to change. Even an infinitely fast analysis stage would save at most about 25% of CPU here. A COOL speedup claim for COOLPilot must be stated per stage, not as a headline number.

Migration steps (operator)

1. Use an AWS account on a paid plan, then subscribe to the COOL product for the chosen generation in AWS Marketplace and accept its terms.
2. Launch: `COOLPILOT_COOL_GEN=2 bash deploy/aws/launch.sh cool` (G2 → c6g.xlarge; `3` → c7g.xlarge; `4` → c8g.xlarge; pass an instance type as the second argument to override). The instance terminates itself after its TTL.
3. `bash deploy/aws/sync.sh`, then run `remote_setup.sh cool` on the instance. It prints the runtime identity, runs the test suite, profiles OpenCV and starts the service.
4. On the **same instance**, also run `remote_setup.sh standard` (a separate venv) to get a same-host baseline, then compare: `python -m experiments.opencv_profile --compare results/profile-standard.json results/profile-cool.json`.
5. Run the evaluation sequence against the COOL service: `python -m experiments.evaluation_runs --api http://127.0.0.1:8769 baseline random agent`.

Measured on COOL (2026-10-27)

Deployment: - **COOL Graviton2 AMI:** `ami-009ab0cb3e3e0acc1` (`COOL-graviton2-v2-prod`, us-east-1), on a **c6g.2xlarge** (Neoverse-N1, 8 vCPU, Ubuntu 24.04), in a separate paid AWS account with the 7-day COOL trial. - **COOL build:** OpenCV **4.12.0** ("4.12.0-dirty"), Custom HAL **KleidiCV 0.5.0 + Carotene**, linked against **Arm Performance Libraries 25.07**, **TBB** parallel framework, NumPy 1.26.4, at `/opt/cool/python_3.12/site-packages`. - The same instance ran two baselines: the standard `opencv-python-headless` **4.12.0.88** wheel (same version, isolates what COOL adds) and our shipped **5.0.0.93** wheel. - Method: `experiments/cool_benchmark.sh` (setup, tests, per-stage profiles at 1 and 8 threads, NanoDet timing, comparisons). Inputs: the three Oxford stereo clips (3 × 271 frames). Evidence: `docs/evidence/cool/`. Cost of the run: about 12 instance-minutes.

COOL executes the core workload. - COOLPilot's own runtime check reported `distribution: COOL`, with accelerators KleidiCV, Carotene, ARMPL and NEON. - **All 62 tests passed** under COOL. - The full service ran under it (`/api/system` → `cool: true`). - COOL and the 4.12 wheel gave **identical measurements and alarms** (0 differences).

Per-stage CPU, same host (A/B > 1 means B is faster):

Comparison	1 OpenCV thread	8 OpenCV threads
4.12 wheel (A) vs **COOL** (B)	total 1.01×; OpenCV analysis 1.01×; decode 0.97×	total **0.90×**; analysis **0.84×** (TBB overhead on our small images)
Standard **5.0.0** (A) vs COOL (B)	total 0.90× (5.0 faster); decode 0.43× (5.0's decoder 2.3× faster)	total 0.82×
NanoDet median ms per frame	4.12 wheel 94.5 · COOL 98.0 · **5.0 new engine 68.9**	4.12 wheel 56.0 · COOL 76.5 · **5.0 new engine 14.7**

Capacity and cost (MEASURED, 2026-10-27). Method: `experiments/cool_capacity.sh`. - **Workload:** 8 camera streams at once, one per vCPU, each with 1 OpenCV thread. Each stream runs the production pipeline (decode, OpenCV analysis, alarm logic; no optional optical flow) on the three Oxford stereo clips, unpaced. - **Capacity:** real-time cameras = total frames/s ÷ the sample's 14.26 fps. - **Prices:** on-demand, us-east-1, from the AWS Pricing API. The COOL software fee is $0 during the trial.

Instance and build	Total frames/s	Real-time cameras	CPU use	Analysis p95 under load	$/h	Cost per 1,000 camera-hours
c6g.2xlarge (Graviton2, Arm), **COOL 4.12**	387.0	27	98.9%	18.5 ms	0.272	**$10.07**
c6g.2xlarge (Graviton2, Arm), **standard OpenCV 5.0.0**	439.9	30	98.8%	21.6 ms	0.272	**$9.07**
c7i.2xlarge (Intel Xeon 8488C, x86), standard OpenCV 5.0.0	483.6	33	98.6%	20.0 ms	0.357	**$10.82**

- **Arm wins on cost.** Standard OpenCV 5 on Graviton2 is 16% cheaper per camera than on x86. COOL on Graviton2 is 7% cheaper than x86, but 11% dearer than standard OpenCV 5 on the same Arm host.
- **COOL had the lowest analysis p95 under load** (18.5 ms), but lower total throughput: decode dominates, and COOL's build decodes more slowly.

- Tests passed on all three: 62 on each Graviton stack, plus the x86 run (`docs/evidence/cool/capacity-*/`).
- An earlier single-stream estimate (`capacity-estimate.json`: about 42, 42 and 60 cameras) was **too optimistic**. With all 8 cores busy, throughput per stream drops (memory bandwidth and decode contention). The measured numbers above replace it.

The autopilot decision for this workload: standard OpenCV 5.0.0 on Graviton2, one thread per camera stream, about 30 cameras per c6g.2xlarge at $9.07 per 1,000 camera-hours. It beats COOL on the same host and x86 on cost. COOL runs it correctly but brings no gain, for three reasons: - **Decoding** (23–44% of CPU) is outside COOL's scope. - Our heaviest calls (`Laplacian`, grey `cvtColor`, `meanStdDev`, `countNonZero`) are not on the KleidiCV HAL list. Only the 0.5× `INTER_AREA` resize and `absdiff` are. - This COOL image is OpenCV 4.12, so it has no OpenCV 5 DNN engine.

Architecture: - The public endpoint is an **arm64 Lambda** with the standard OpenCV 5.0.0 wheel. It is serverless, read-only, and serves uploads through S3. - COOL runs on **EC2 Graviton**, because COOL ships as a Marketplace AMI and Lambda cannot use one. - Evaluation and benchmarking ran on EC2 Graviton (t4g.small, standard wheel; c6g.2xlarge, COOL). - **x86 appears only as a measured baseline** (c7i.2xlarge). Nothing in the deployed system runs on x86. Arm handles both serving (Lambda) and the camera fleet tier (EC2 Graviton).

Bug found and fixed: `remote_setup.sh cool` put only our own dependencies on `PYTHONPATH`, so the vendor `cv2` was not found. It now adds `/opt/cool/python_3.x/site-packages`, the layout measured on the AMI.

Evidence checklist (for the Best Use of COOL award)

All items must come from the same COOL instance:

- [ ] COOL product, version and Graviton generation, plus the instance type
- [ ] `/api/system` shows `distribution: "COOL"` and `cv2.__file__` under `/opt/cool`
- [ ] `results/build-cool.txt` (full `cv2.getBuildInformation()`)
- [ ] `opencv_profile --compare` standard vs COOL, on the same host, per stage
- [ ] A completed run on `oxford-camera-holdout` with the COOL environment recorded in every experiment
- [ ] An agent run whose search includes `threads`, showing whether more threads helped under COOL